Smart Schedule.

From a busy week
to realistic work availability.

A WEEK WITH MANY ROLES

09:00 Classes

13:00 Job A

18:00 Personal time

When can I actually work?

THE PROJECT IDEA

One week.
All your commitments.

01

Bring it together

Classes, multiple jobs,
and personal time.

02

See conflicts

Find commitments
that overlap.

03

Share availability

Offer workable hours
without private details.

Planned

> Empty calendar space ≠ willingness to work.

Willing hours − commitments = work availability.



WHAT THE EMPLOYER RECEIVES

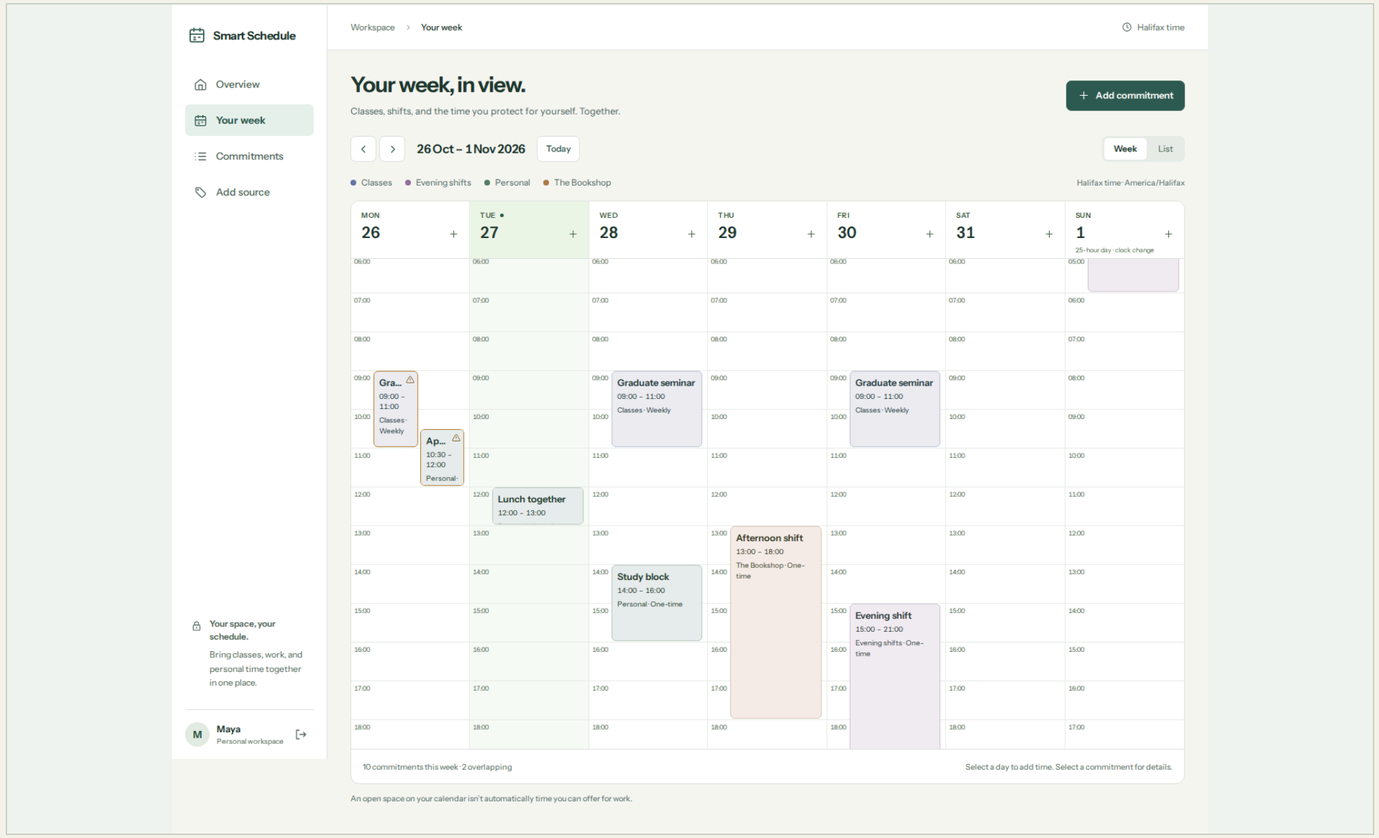
Monday

08:00–10:00

14:00–20:00

Available times only.
No class or personal details.

The calendar is working.



Current app · Fictional sample schedule

Working interface

Private accounts & commitment entry

Weekly calendar, overnight shifts, one-time save warnings.

Backend complete

Recurring schedules

Future edits and one-off exceptions. Entry forms are next.

189 PostgreSQL tests passed

Latest recorded verification

HOW IT IS BUILT

Keep scheduling rules in one place.

INTERFACE

Django templates

Bootstrap + focused JavaScript

→

BUSINESS LOGIC

Scheduling services

Recurrence · conflicts · time rules

→

STORAGE

PostgreSQL

Private, structured schedule data

THE CHALLENGE

Overnight shifts. Daylight-saving changes.
Schedule changes that preserve earlier weeks.

Python · Django · PostgreSQL · pytest

NEXT STEPS

Turn the schedule into an answer.

01 Recurring entry & conflict integration

02 Working windows & availability calculation

03 Copy availability for an employer

Later: natural-language entry & Google Calendar integration.

“When can I actually work?”